

Functional Programming

Project: Hoodle (part 2)

Assistant: Noah Van Es
`noah.van.es@vub.be`

The final grade of the course is determined by an oral exam and a project (each counts for 50% of the total grade for the course). The project consists out of two parts. A meaningful solution for both parts has to be submitted in order to get a grade. This document describes the second part of the project.

Project Description

The goal of this assignment is to extend the Hoodle application of the first assignment by setting up a server which exposes its functionality. Importantly, the server should be able to process multiple requests in parallel (without introducing race conditions).

Practical Information

Submission. The deadline for this assignment is **June 8, 2026 (23:59)**. You submit all your code and other materials in a single ZIP-file through the Canvas platform. Optionally, you may include a `README.md` file in your archive to clarify certain aspects of your implementation that warrant further explanation (e.g., the structure of your project).

Grading. Note that your project will not only be graded on functionality, but also on the quality of your implementation (programming style, performance considerations, ...). In particular, **you will be graded on the correct use of FP concepts and abstractions discussed during the lectures and exercise sessions.**

Individual Work. This project is to be made strictly individually and on your own. This means that you must create your project on an independent basis and you must be able to explain your work, reimplement it under supervision, and defend your solution. Copying work (code, text, etc.) from or sharing it with third parties (e.g., fellow students, generative AI tools, websites, GitHub, etc.) is not allowed. Electronic tools are used to compare all submissions with online resources and with each other, even across academic years. Any suspicion of plagiarism will be reported to the Dean of Faculty

without delay. Both user and provider of such code will be reported and will be dealt with according to the plagiarism rules of the examination regulations (cf. OER, Article 118). The dean may decide on (a combination of) the disciplinary sanctions, ranging from 0/20 on the course to a prohibition from (re-)enrolling for one or multiple academic years (cf. OER, Article 118§5). The use of any third-party code, other than the libraries listed below, is forbidden. The use of generative AI is not allowed in any form. Contact us if you are in doubt as to whether or not something would be considered plagiarism.

Available Resources. In case you have any questions or need further clarification to complete the project, you can contact the assistant during the exercise sessions or by mail (noah.van.es@vub.be). You are allowed to use third-party Haskell libraries, as well as Haskell language extensions discussed during the exercise sessions.

Project Assignment

Your code should support the functionalities described below. **Ensure that your project can be compiled using version 9.6.7 of the GHC compiler¹.**

For this assignment, you will have to create a TCP server that implements the protocol detailed in Table 1. Your server should handle TCP requests concurrently using the module `Network.TCP` and the function `Control.Concurrent.forkIO`. Each TCP connection is expected to handle exactly one client request and one server response, after which it should be closed. Data race conditions should be handled using software transactional memory from the module `Control.Concurrent.STM`, as seen in the lectures.

To manage access rights, requests should always embed a user login. If the user identifier does not exist for the requested role or if the password does not match, the server should return the message `wrong-login`.

Concretely, the requests that your server should handle are listed below.

add-user : enables the administrator to add a user to the system. If the user executing this request is not an administrator, the server should return the message `not-permitted`. If the requested user identifier is already taken, the server should return the message `id-taken`. If the requested user identifier is available, it should return a randomly generated password of at least 4 characters. The administrator login should be passed as command-line arguments during startup.

```
=> add-user admin:1234 alice
<= ok k74X
=> add-user admin:1234 alice
<= id-taken
=> add-user alice:k74X bob
<= not-permitted
```

change-password : enables users to change their password.

¹<https://www.haskell.org/ghc/>

```
=> change-password alice:k74X foobar
<= ok
```

add-hoodle : enables users to create new hoodles. Unlike in the first part of the programming assignment, hoodle slots are allowed to overlap. However, if the hoodle contains invalid timeslots (i.e., where the starting time of the slot does not occur before the ending time of the slot), the server should return the message **invalid-hoodle**. If another hoodle already exists with the same name, the server should return the message **id-taken**.

```
=> add-hoodle alice:foobar fp-party [
  2026-01-04T20:00+01:00 / 2026-01-04T22:00+01:00,
  2026-01-04T12:00+01:00 / 2026-01-04T14:00+01:00,
]
<= ok
```

edit-hoodle : enables users to edit a hoodle they created. If a user attempts to edit a hoodle that they did not create themselves, the server should return the message **not-permitted**. If no hoodle exists with the given name, the server should return the message **no-such-id**.

```
=> edit-hoodle alice:foobar fp-party [
  2026-01-04T20:00+01:00 / 2026-01-04T22:00+01:00,
  2026-01-04T16:00+01:00 / 2026-01-04T18:00+01:00,
  2026-01-04T12:00+01:00 / 2026-01-04T14:00+01:00
]
<= ok
```

remove-hoodle : enables a user to remove a hoodle they created. If a user attempts to remove a hoodle that they did not create themselves, the server should return the message **not-permitted**. If no such hoodle exists, the server should return the message **no-such-id**.

```
=> remove-hoodle alice:foobar fp-party
<= ok
```

get-hoodle : enables users to get the hoodle associated with a name. If no hoodle with the given name exists, the server should return the message **no-such-id**. The timeslots should be sorted by starting time.

```
=> get-hoodle fp-party
<= ok [
  2026-01-04T12:00+01:00 / 2026-01-04T14:00+01:00,
  2026-01-04T20:00+01:00 / 2026-01-04T22:00+01:00
]
```

register : enables users to register to a hoodle. If no hoodle with the given name exists, the server should return the message **no-such-id**. If the user is already registered, the server should return the message **already-registered**.

```
=> register alice:foobar fp-party
<= ok
```

unregister : enables users to unregister from a hoodle. If no hoodle with the given name exists, the server should return the message **no-such-id**. If the user is not yet registered, the server should return the message **not-registered**.

```
=> unregister alice:foobar fp-party  
<= ok
```

toggle : enables registered users to toggle a timeslot. If no hoodle with the given name exists, the server should return the message **no-such-id**. If the user is not registered to the hoodle, the server should return the message **not-registered**. If the given slot is not part of the hoodle, the server should return the message **invalid-slot**.

```
=> toggle alice:foobar fp-party 2026-01-04T20:00+01:00/2026-01-04T22:00+01:00  
<= ok
```

optimal-schedule : enable a user to get the current optimal schedule (as defined in the first programming assignment) for all hoodles in the system. The optimal schedule is computed taking all hoodles of the system into account, but the server response should contain only those to which the user is registered. If there is no schedule possible, the server should return the message **no-possible-schedule**.

```
=> optimal-schedule alice:foobar  
<= {fp-party : 2026-01-04T20:00+01:00/2026-01-04T22:00+01:00}
```

Grammar Rule		Comments
REQUEST	→ add-user LOGIN TOKEN change-password LOGIN TOKEN get-hoodle LOGIN TOKEN add-hoodle LOGIN TOKEN HOODLE edit-hoodle LOGIN TOKEN HOODLE remove-hoodle LOGIN TOKEN register LOGIN TOKEN unregister LOGIN TOKEN toggle LOGIN TOKEN SLOT optimal-schedule LOGIN	The administrator LOGIN adds the user TOKEN to the system. Change the password of LOGIN to TOKEN. User LOGIN gets the hoodle with the name TOKEN. User LOGIN adds the hoodle TOKEN with the slots given in HOODLE. User LOGIN edits the hoodle TOKEN, replacing the slots with those given in HOODLE. User LOGIN removes the hoodle TOKEN. User LOGIN registers to the hoodle TOKEN. User LOGIN unregisters from the hoodle TOKEN. User LOGIN toggles the slot SLOT in the hoodle with name TOKEN. User LOGIN gets the optimal schedule for all hoodles in the system.
RESPONSE	→ wrong-login ok TOKEN ok HOODLE ok SCHEDULE ok id-taken not-permitted invalid-hoodle no-such-id no-such-slot not-registered already-registered no-possible-schedule	Applicable for all requests. Applicable for: add-user. Applicable for: get-hoodle. Applicable for: optimal-schedule. Applicable for all requests other than add-user, get-hoodle and optimal-schedule. Applicable for: add-user, add-hoodle. Applicable for: add-user, edit-hoodle and remove-hoodle. Applicable for: add-hoodle. Applicable for: get-hoodle, edit-hoodle, remove-hoodle, register, unregister and toggle. Applicable for: toggle. Applicable for: toggle and unregister. Applicable for: register. Applicable for: optimal-schedule.
LOGIN	→ TOKEN:TOKEN	A login is an identifier paired with a password.
HOODLE	→ [SLOTS] → [] SLOTS → SLOT , SLOTS SLOT	A hoodle is a list of timeslots.
SLOT	→ TIME / TIME	A timeslot is defined by a start time and an end time.
SCHEDULE	→ { HOODLES } → { }	A schedule is a mapping from hoodle names to timeslots.
HOODLES	→ TOKEN : SLOT , HOODLES TOKEN : SLOT	

Table 1: Client-server exchange protocol. Any white space in the grammar rules can be replaced by a combination of white spaces, tabs and newline characters. TOKEN is any group of letters uninterrupted by white spaces, tabs and newline characters. TIME follows the ISO8601 format – e.g: 2026-07-01T10:24+01:00.